

MINI MIPS 16bits

lpabarbosa

October 2020

1 Introdução

Neste documento vamos estudar vamos estudar uma arquitetura de 16 bits baseada no MIPS, denominada MINI MIPS 16bits. Estudaremos uma versão de ciclo único do caminho de dados implementada usando o simulador Logisim, que será enviado juntamente com este documento. Nas seções seguintes conheceremos mais dessa arquitetura, de seu conjunto de instruções, bem como alguns dos detalhes de sua implementação e funcionamento durante a execução de instruções.

2 Conjunto de instruções

Nome	Formato	Operação		Opcode/Funct(hex.)
add	R	$R[rd] = R[rs] + R[rt]$	(1)	0/0
addi	I	$R[rt] = R[rs] + \text{SignExtImm}$	(1,2)	8/-
and	R	$R[rd] = R[rs] \& R[rt]$	-	0/4
beq	I	$\text{if}(R[rs] == R[rt])$ $PC = PC + 2 + \text{BranchAddr}$	(4)	4/-
j	J	$PC = \text{JumpAddr}$	(5)	2/-
lw	I	$R[rt] = M[R[rs] + \text{SignExtImm}]$	(2)	3/-
nand	R	$R[rd] = \sim(R[rs] \& R[rt])$	-	0/1
nor	R	$R[rd] = \sim(R[rs] R[rt])$	-	0/7
or	R	$R[rd] = R[rs] R[rt]$	-	0/5
slt	R	$R[rd] = (R[rs] < R[rt]) ? 1 : 0$	-	0/2
sw	I	$M[R[rs] + \text{SignExtImm}] = R[rt]$	(2)	a/-
sub	R	$R[rd] = R[rs] - R[rt]$	(1)	0/3

Table 1: Conjunto de instruções do MINI MIPS 16bits

- (1) Pode causar exceção por overflow
- (2) $\text{SignExtImm} = \{ 10\{\text{immediate}[5]\}, \text{immediate} \}$
- (3) $\text{ZeroExtImm} = \{ 10\{1b'0\}, \text{immediate} \}$
- (4) $\text{BranchAddr} = \{ 9\{\text{immediate}[5]\}, \text{immediate}, 1'b0 \}$
- (5) $\text{JumpAddr} = \{ PC+2[15:13], \text{address}, 1'b0 \}$

R-format	op	rs	rt	rd	funct	(não tem shamt)
	4	3	3	3	3	bits

I-format	op	rs	rt	immd	
	4	3	3	6	bits

J-format	op	address	
	4	12	bits

3 O circuito de controle

Os circuitos de controle são responsáveis pela configuração de dos multiplexadores, que controlam o fluxo dos dados durante a execução, do banco de registradores, da ALU, da memória de dados, etc.

3.1 O circuito de controle geral

O controle geral recebe os quatro bits do campo *opcode* da instrução como entradas e produz como saídas os dez bits de controle, incluindo os bits *ALUop1* e *ALUop0* que serão usados pelo controle da ALU. Estes bits servem para determinar se o campo *funct* deve ser considerado ou não na produção dos sinais, *Ainv*, *Bneg*, *op1* e *op0*, os quais, por sua vez, servem para selecionar a operação desejada para a ALU.

Além dos sinais mencionados temos ainda:

- *RegDst* - determina qual dos registradores *rt* ou *rd* será usado como destino do resultado da operação,
- *Jump* - indica que está sendo executada uma instrução do tipo *jump*,
- *Branch* - indica que está sendo executada uma instrução de desvio, por exemplo *beq*,
- *MemRead* - configura a memória de dados para leitura,
- *MemtoReg* - controla se a informação que vai para a entradas *Write Data* do banco de registradores vem da saída da memória ou da saída da ALU,
- *MemWrite* - configura a memória de dados para escrita.

A menos que indicado o contrário, todos os sinais usam lógica positiva, ou seja, o comportamento descrito é obtido com o sinal em questão assumindo o valor 1.

A seguir temos a Tabela 2 que define o funcionamento do controle geral. Nela estão indicadas as entradas e saídas do circuito e o modo como elas se relacionam, permitindo, desta forma, o projeto do circuito de controle geral.

<i>OpCode3</i>	0	0	1	0	1	0	Input
<i>OpCode2</i>	0	0	0	1	0	0	
<i>OpCode1</i>	0	1	1	0	0	1	
<i>OpCode0</i>	0	1	0	0	0	0	
Instruction	R-format	lw	sw	beq	addi	j	
<i>RegDst</i>	1	0	X	X	0	X	Output
<i>JUMP</i>	0	0	0	0	0	1	
<i>Branch</i>	0	0	0	1	0	0	
<i>MemRead</i>	0	1	0	0	0	0	
<i>MemtoReg</i>	0	1	X	X	0	X	
<i>ALUOp1</i>	1	0	0	0	1	X	
<i>ALUOp0</i>	0	0	0	1	1	X	
<i>MemWrite</i>	0	0	1	0	0	0	
<i>ALUSrc</i>	0	1	1	0	1	X	
<i>RegWrite</i>	1	1	0	0	1	0	

Table 2: Tabela para o projeto do controle geral

O projeto consiste em usar mapas de Veitch-Karnaugh e álgebra de Boole para determinar o circuito para cada um dos sinais de saída. Usaremos as abreviações, *Op_i*, *i* = 0, 1, 2, 3., para representar os bits do *OpCode* nos mapas e expressões.

Para o sinal *RegDst*, temos:

		<i>Op₁Op₀</i>			
<i>Op₃Op₂</i>		00	01	11	10
	00	1			X
	01	X			
	11				
	10				X

Para os sinais *Jump*, *Branch* e *MemRead*, é possível escrever diretamente suas expressões, pois possuem apenas um minitermo:

$$RegDst = \overline{Opc_3} \overline{Opc_1} \overline{Opc_0} \quad (1)$$

$$Jump = \overline{Opc_3} \overline{Opc_1} Opc_1 \overline{Opc_0} \quad (2)$$

$$Branch = \overline{Opc_3} Opc_2 \overline{Opc_1} \overline{Opc_0} \quad (3)$$

$$MemRead = \overline{Opc_3} \overline{Opc_2} Opc_1 Opc_0 \quad (4)$$

Para o sinal *MemtoReg*, temos:

		<i>Opc₁Opc₀</i>			
<i>Opc₃Opc₂</i>		00	01	11	10
	00			1	X
	01	X			
	11				
	10				X

$$MemtoReg = \overline{Opc_3} \overline{Opc_2} Opc_1 \quad (5)$$

Para os sinais *ALUop1* e *ALUop0*, fizemos:

		<i>Opc₁Opc₀</i>			
<i>Opc₃Opc₂</i>		00	01	11	10
	00	1			X
	01				
	11				
	10	1			

$$ALUop1 = \overline{Opc_2} \overline{Opc_1} \overline{Opc_0} \quad (6)$$

		<i>Opc₁Opc₀</i>			
<i>Opc₃Opc₂</i>		00	01	11	10
	00				X
	01	1			
	11				
	10	1			

$$ALUop0 = \overline{Opc_3} \overline{Opc_2} \overline{Opc_1} \overline{Opc_0} + Opc_3 \overline{Opc_2} \overline{Opc_1} \overline{Opc_0} \quad (7)$$

$$= Branch + Opc_3 \overline{Opc_2} \overline{Opc_1} \overline{Opc_0} \quad (8)$$

O sinal *MemWrite* é obtido direto da tabela:

$$MemWrite = Opc_3 \overline{Opc_2} \overline{Opc_1} \overline{Opc_0} \quad (9)$$

Para o sinal *ALUsrc*, temos:

$Opc_3 Opc_2$	$Opc_1 Opc_0$			
	00	01	11	10
00			1	X
01				
11				
10	1			1

$$ALUsrc = Opc_3 \overline{Opc_2} \overline{Opc_0} + \overline{Opc_3} \overline{Opc_2} Opc_1 \quad (10)$$

$$= Opc_3 \overline{Opc_2} \overline{Opc_0} + MemtoReg \quad (11)$$

E finalmente, para *RegWrite*, obtemos:

$$RegWrite = \overline{Opc_2} \overline{Opc_1} \overline{Opc_0} + \overline{Opc_3} \overline{Opc_2} Opc_1 Opc_0 \quad (12)$$

$$= ALUop1 + MemRead \quad (13)$$

O resultado final está ilustrado na Figura 1.

3.2 Controle da ALU

O controle da ALU é responsável por programar a ALU para realizar a operação correta para cada instrução do programa. Esse circuito recebe dois bits do controle geral, denominados *ALUop1* e *ALUop0*, bem como os três bits do campo *funct* como entradas e produz como saídas os quatro bits de controle da ALU *Ainv*, *Bneg*, *op1* e *op0*.

Na Tabela 3

Formato	Instrução	ALUOP		FUNCT			Operação da ALU	ALU Control Input			
		<i>ALUOp1</i>	<i>ALUOp0</i>	<i>F2</i>	<i>F1</i>	<i>F0</i>		<i>Ainv</i>	<i>Bneg</i>	<i>op1</i>	<i>op0</i>
I-type	addi	1	1	X	X	X	add	0	0	1	0
I-type	lw	0	0	X	X	X	add	0	0	1	0
I-type	sw	0	0	X	X	X	add	0	0	1	0
I-type	beq	0	1	X	X	X	subtract	0	1	1	0
R-type	add	1	0	0	0	0	add	0	0	1	0
R-type	sub	1	0	0	1	1	subtract	0	1	1	0
R-type	and	1	0	1	0	0	AND	0	0	0	0
R-type	or	1	0	1	0	1	OR	0	0	0	1
R-type	nor	1	0	1	1	1	AND (ent. barradas)	1	1	0	0
R-type	nand	1	0	0	0	1	OR (ent. barradas)	1	1	0	1
R-type	slt	1	0	0	1	0	set on less than	0	1	1	1

Table 3: Tabela com as informações sobre o funcionamento do controle da ALU.

Para o projeto fica melhor assim:

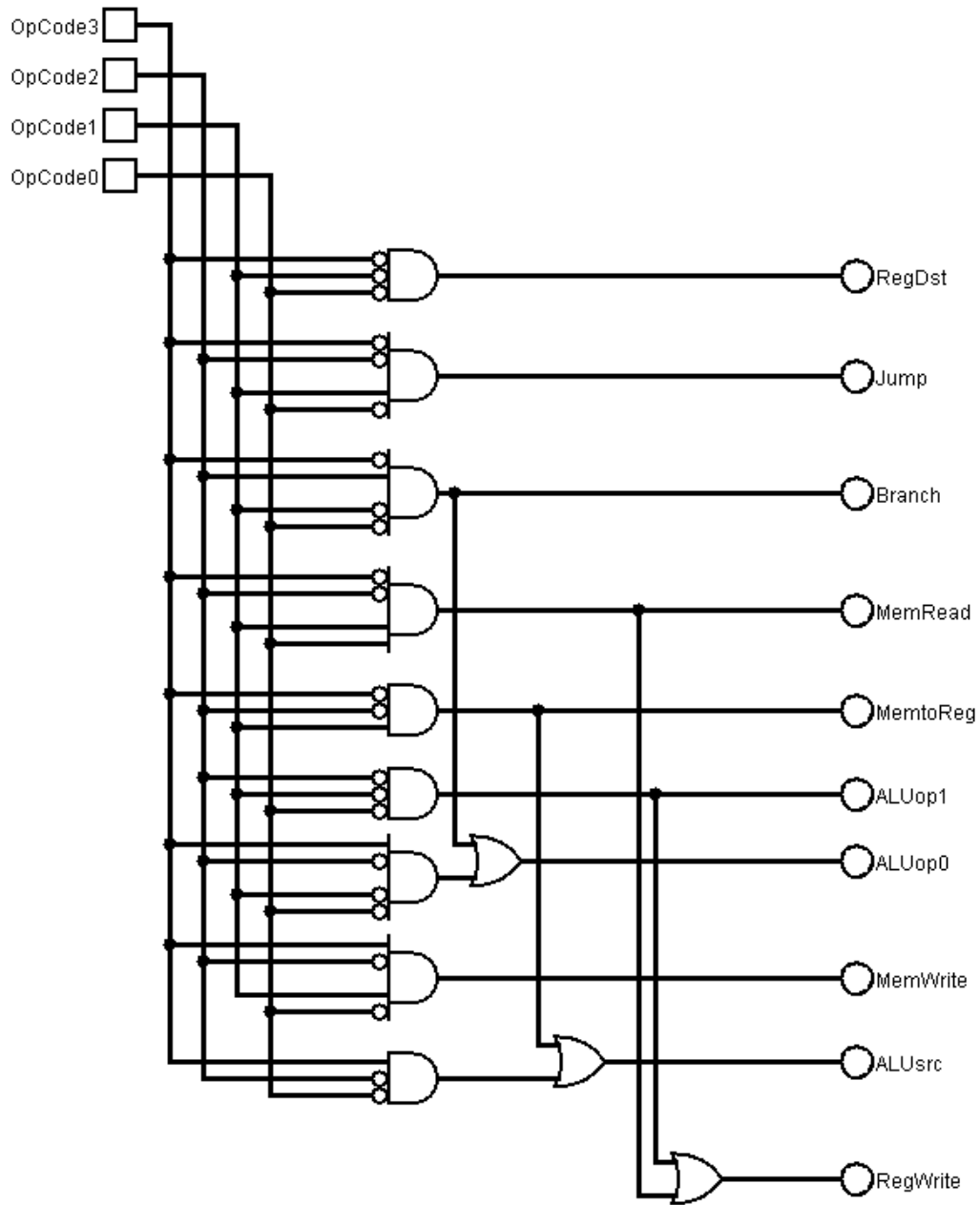


Figure 1: Circuito que implementa o controle geral do MINI MIPS

<i>ALUOp1</i>	1	0	0	0	1	1	1	1	1	1	1	Input
<i>ALUOp0</i>	1	0	0	1	0	0	0	0	0	0	0	
F2	X	X	X	X	0	0	1	1	1	0	0	
F1	X	X	X	X	0	1	0	0	1	0	1	
F0	X	X	X	X	0	1	0	1	1	1	0	
Ainv	0	0	0	0	0	0	0	0	1	1	0	Output
Bneg	0	0	0	1	0	1	0	0	1	1	1	
op1	1	1	1	1	1	1	0	0	0	0	1	
op0	0	0	0	0	0	0	0	1	0	1	1	

$$Ainv = A_1 \overline{A_0} F_2 F_1 F_0 + A_1 \overline{A_0} \overline{F_2} \overline{F_1} F_0 \quad (14)$$

$$= A_1 \overline{A_0} F_0 (F_2 \odot F_1) \quad (15)$$

$$Bneg = \overline{A_1} A_0 + A_1 \overline{A_0} \overline{F_2} F_1 F_0 + A_1 \overline{A_0} F_2 F_1 F_0 + A_1 \overline{A_0} \overline{F_2} \overline{F_1} F_0 + A_1 \overline{A_0} \overline{F_2} F_1 \overline{F_0} \quad (16)$$

$$= \overline{A_1} A_0 + A_1 \overline{A_0} (\overline{F_2} F_1 F_0 + F_2 F_1 F_0 + \overline{F_2} \overline{F_1} F_0 + \overline{F_2} F_1 \overline{F_0}) \quad (17)$$

$$= \overline{A_1} A_0 + A_1 \overline{A_0} (F_1 F_0 + \overline{F_2} F_0 + \overline{F_2} F_1) \quad (18)$$

$$op1 = A_1 A_0 + \overline{A_1} \overline{A_0} + \overline{A_1} A_0 + A_1 \overline{A_0} \overline{F_2} F_1 F_0 + A_1 \overline{A_0} \overline{F_2} F_1 \overline{F_0} + A_1 \overline{A_0} \overline{F_2} \overline{F_1} \overline{F_0} \quad (19)$$

$$= \overline{A_1} + A_0 + A_1 \overline{A_0} \overline{F_2} F_1 + A_1 \overline{A_0} \overline{F_2} \overline{F_1} \overline{F_0} \quad (20)$$

$$= \overline{A_1} + A_0 + A_1 \overline{A_0} \overline{F_2} (F_1 + \overline{F_1} \overline{F_0}) \quad (21)$$

$$= \overline{A_1} + A_0 + \overline{F_2} (F_1 + \overline{F_0}) \quad (22)$$

$$= \overline{A_1} + A_0 + \overline{F_2} F_1 + \overline{F_2} \overline{F_0} \quad (23)$$

$$(24)$$

$$op0 = \overline{A_1} A_0 (F_2 \overline{F_1} F_0 + \overline{F_2} \overline{F_1} F_0 + \overline{F_2} F_1 \overline{F_0}) \quad (25)$$

$$= \overline{A_1} A_0 (\overline{F_1} F_0 + \overline{F_2} F_1 \overline{F_0}) \quad (26)$$

4 ALU (Arithmetic and Logic Unity)

Nesta seção veremos como implementar a ALU do MINI MIPS 16bits. Veremos primeiramente como fazer uma ULA de 1 bit e depois como combinar 16 delas para obter a ULA de completa. Veremos também que a ULA de 1 bit para o bit 15 é ligeiramente diferente das demais.

4.0.1 ALU para os bits 0 a 14

Para os bits 0 a 14 teremos a configuração mostrada na Figura 3, nela vemos a possibilidade de realizar as operações: inversão dos bits de entrada de forma independente, and, or e adição. A subtração pode ser realizada colocando *Cin* em 1 e invertendo o bit da entrada B, usando *Binv* igual a 1 (tente explicar o motivo). Pode-se ainda implementar as operações nand e nor usando a propriedade de De Morgan e a negação é obtida fazendo a nor de uma das entradas enquanto a outra vale zero. O MUX na saída seleciona qual dos resultados computados vai aparecer na saída, se é o resultado da porta lógica and, da porta lógica or, ou do circuito somador completo.

4.0.2 ALU para o bit 15 com detecção de *overflow*

Para o bit 15 temos a adição de um detector de *overflow* para as operações com operandos representados em complemento de 2, aqui optou-se por fazer o ou-exclusivo do *Cin* com o *Cout* do bit 15 conforme indicado no circuito.

Por fim, na Figura 5, temos o circuito completo da ALU. Observe que para o bit 0, unimos o sinal *Binv* com o *Cin* para formar o sinal *Bneg*. Colocando *Bneg* em 1, fazemos o complemento de 2 da entrada B e a ALU realiza $A + C_1(B) + 1 = A + C_2(B) = A - B$, ou seja uma subtração. Há também a adição do circuito que detecta quando o resultado é zero. Esse sinal é utilizado na execução da instrução *beq* juntamente com o sinal de controle *Branch*.

5 Banco de registradores

Nesta seção temos a implementação do banco de registradores do MINI MIPS 16bits, que possui 8 registradores mais o PC que é representado à parte. Vemos o uso de decodificadores de endereços e multiplexadores para controlar quais informações serão apresentadas nas saídas *Read Data 1* e *Read Data 2* e em qual registrador os dados presentes na entrada *Write Data* será guardado.

6 MINI MIPS 16bits de ciclo único

Por fim, temos o caminho dados do MINI MIPS 16 bits de ciclo único, o que significa que cada instrução é realizada em apenas um ciclo de *clock*. Nele, os diversos elementos mencionados anteriormente se unem para realizar as instruções dos programas da forma descrita na Tabela 1.

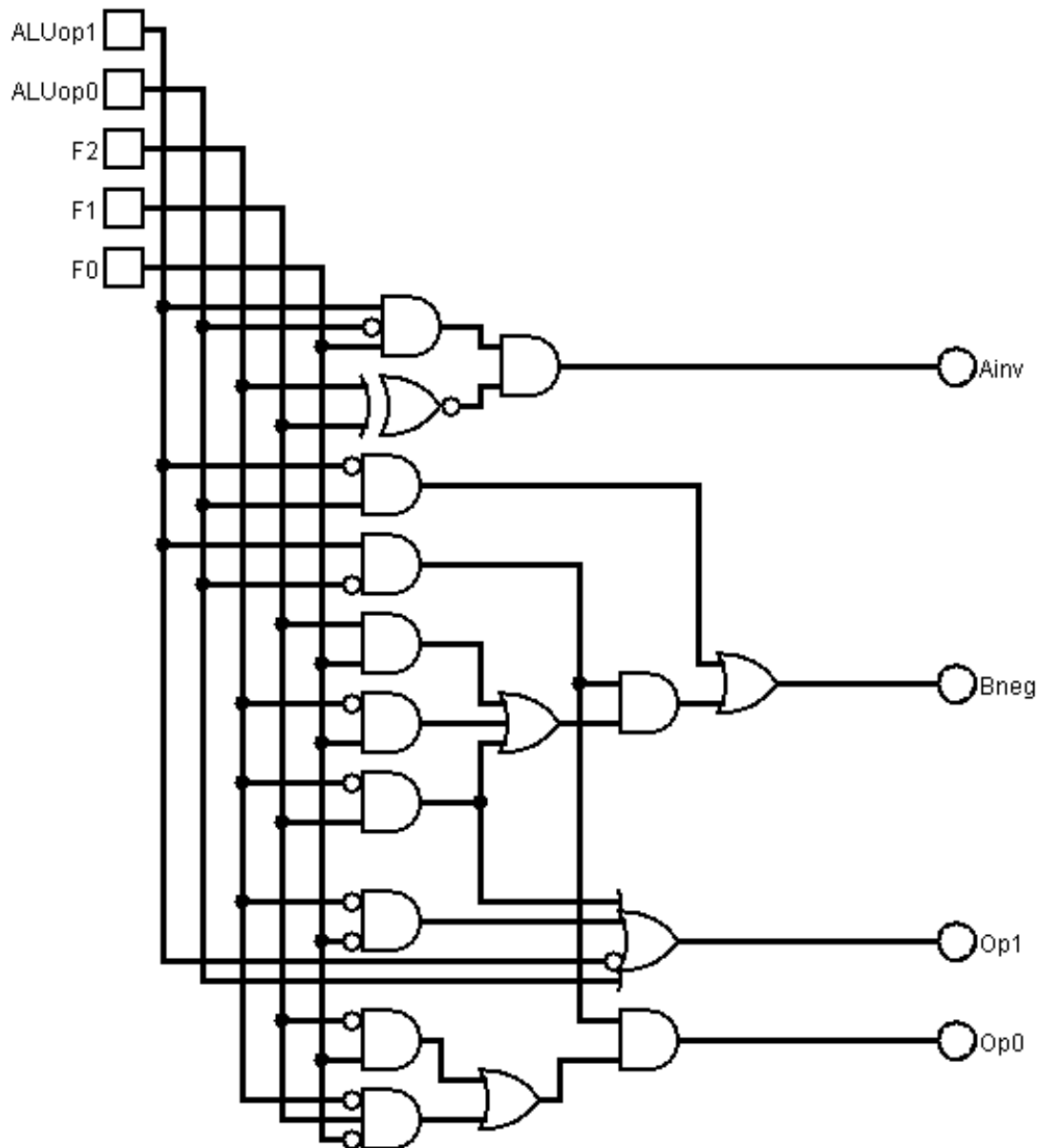


Figure 2: Circuito que implementa o controle da ALU do MINI MIPS

7 Exemplos

7.1 Exemplo 1

O exemplo 1 consiste em carregar dois operandos da memória para os registradores, adicioná-los e depois colocar o resultado na memória. Considere todos os registradores do MINI MIPS 16 bits iguais a 0 no início da execução. E a memória de dados com os valores 0x005 e 0x000a nas posições 0x0000 e 0x0002, respectivamente.

Memória de instruções:

Addr	assembly	campos (bin)	código de máquina	conteúdo da mem. (hex)
0x0000	-> lw \$2, 0(\$1)	-> 0011 001 010 000000	-> 0011 0010 1000 0000	-> 0x3280
0x0002	-> lw \$3, 2(\$1)	-> 0011 001 011 000010	-> 0011 0010 1100 0010	-> 0x32c2
0x0004	-> add \$4, \$2, \$3	-> 0000 010 011 100 000	-> 0000 0100 1110 0000	-> 0x04e0
0x0006	-> sw \$4, 4(\$1)	-> 1010 001 100 000100	-> 1010 0011 0000 0100	-> 0xa304

Memória de dados (ao final da execução):

Addr	Conteúdo (hex)
0x0000	-> 0x0005 (operando 1)
0x0002	-> 0x000a (operando 2)
0x0004	-> 0x000f (resultado)

8 Práticas

- (1) Execute o código do Exemplo 1.
- (2) Adicione as instruções `sub $5, $2, $3` e uma instrução `store word` para guardar o resultado da subtração na próxima posição de memória disponível.
- (3) Modifique o código para efetuar a adição se os conteúdos dos registradores `$2` e `$3` forem iguais e a subtração no caso contrário. Teste as duas possibilidades.
- (4) Escreva um laço para colocar os valores de 0 a 5 a partir da posição de memória `0x000a`.
- (5) Explique todos valores que aparecem destacados no simulador do MINI MIPS 16bits durante essas práticas. Faça isso para cada instrução.

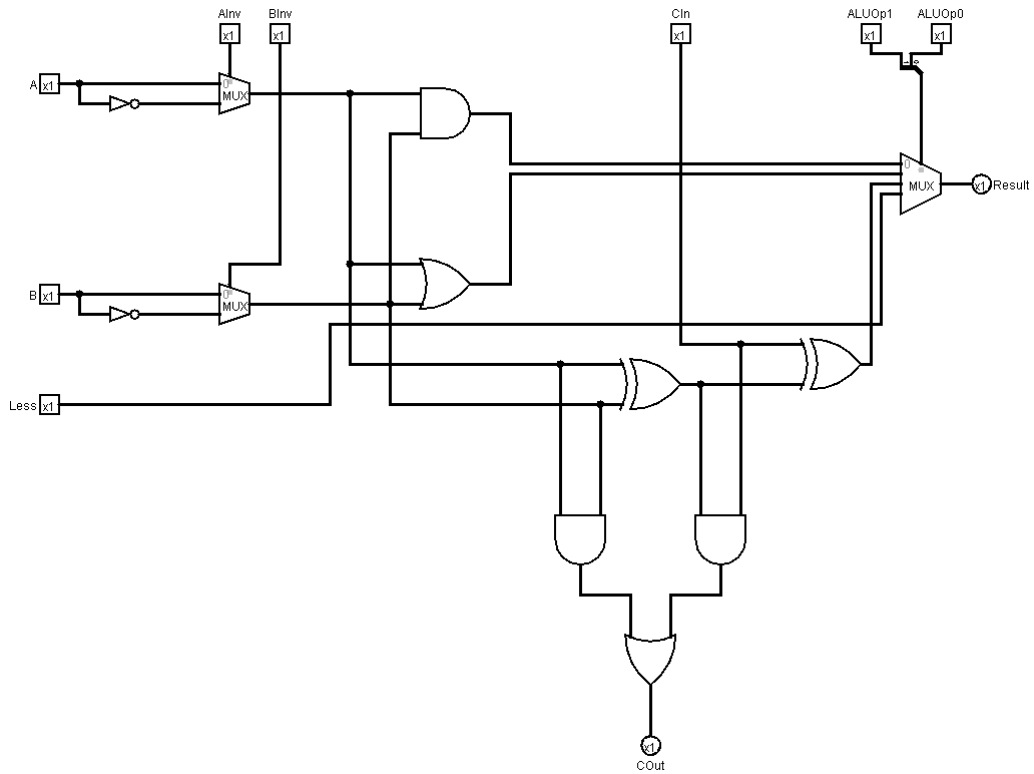


Figure 3: Circuito que implementa os bits 0 a 14 da ALU do MINI MIPS 16bits

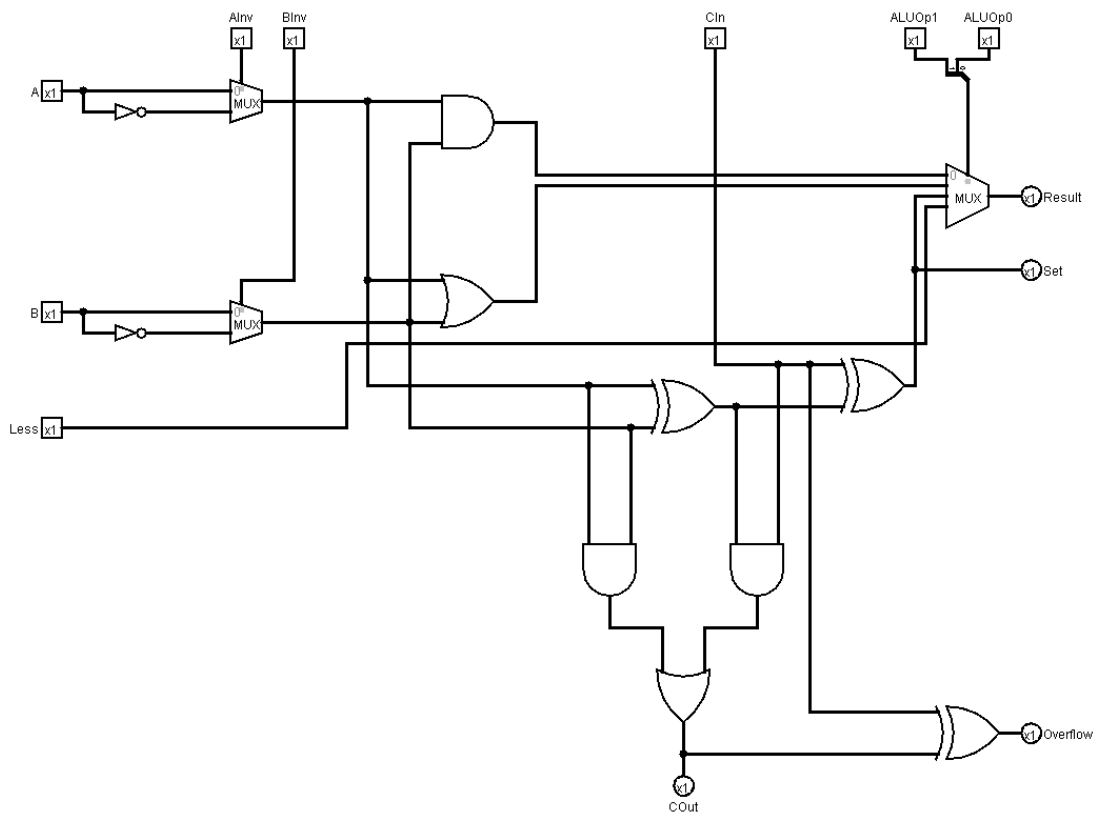


Figure 4: Circuito que implementa o bit 15 da ALU do MINI MIPS 16bits

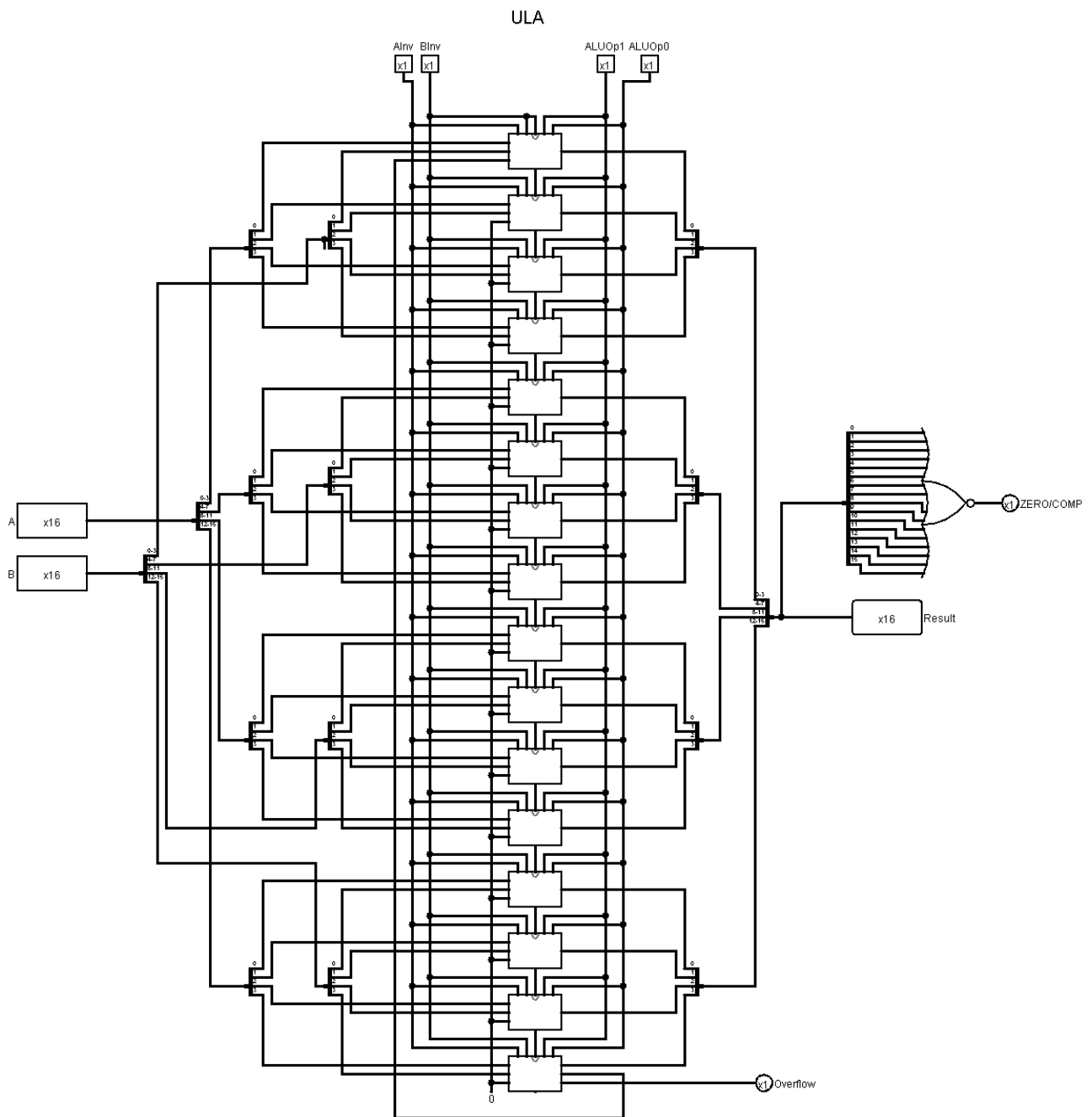


Figure 5: Circuito que implementa a ALU do MINI MIPS 16bits

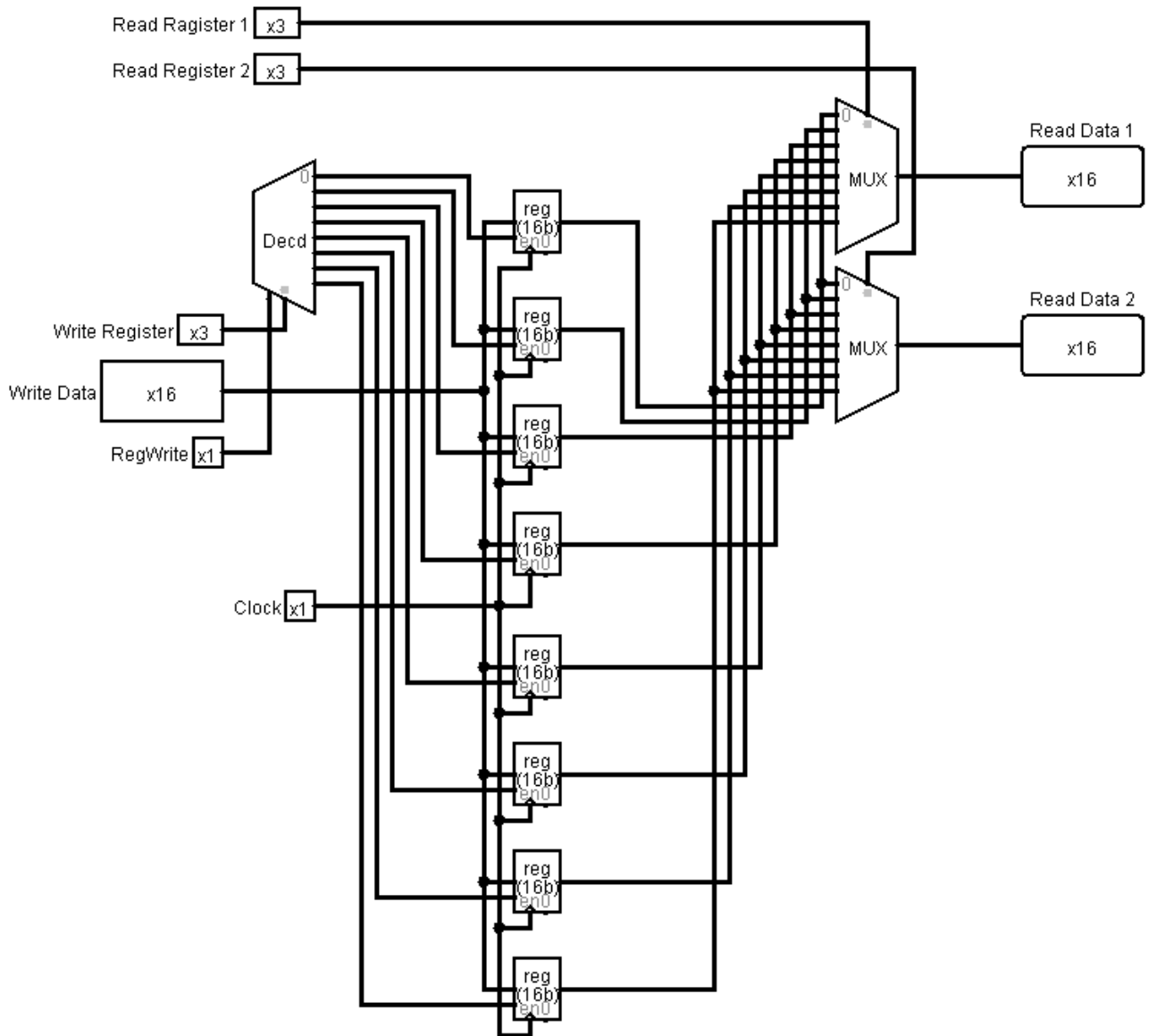


Figure 6: Circuito que implementa o banco de registradores do MINI MIPS 16bits.

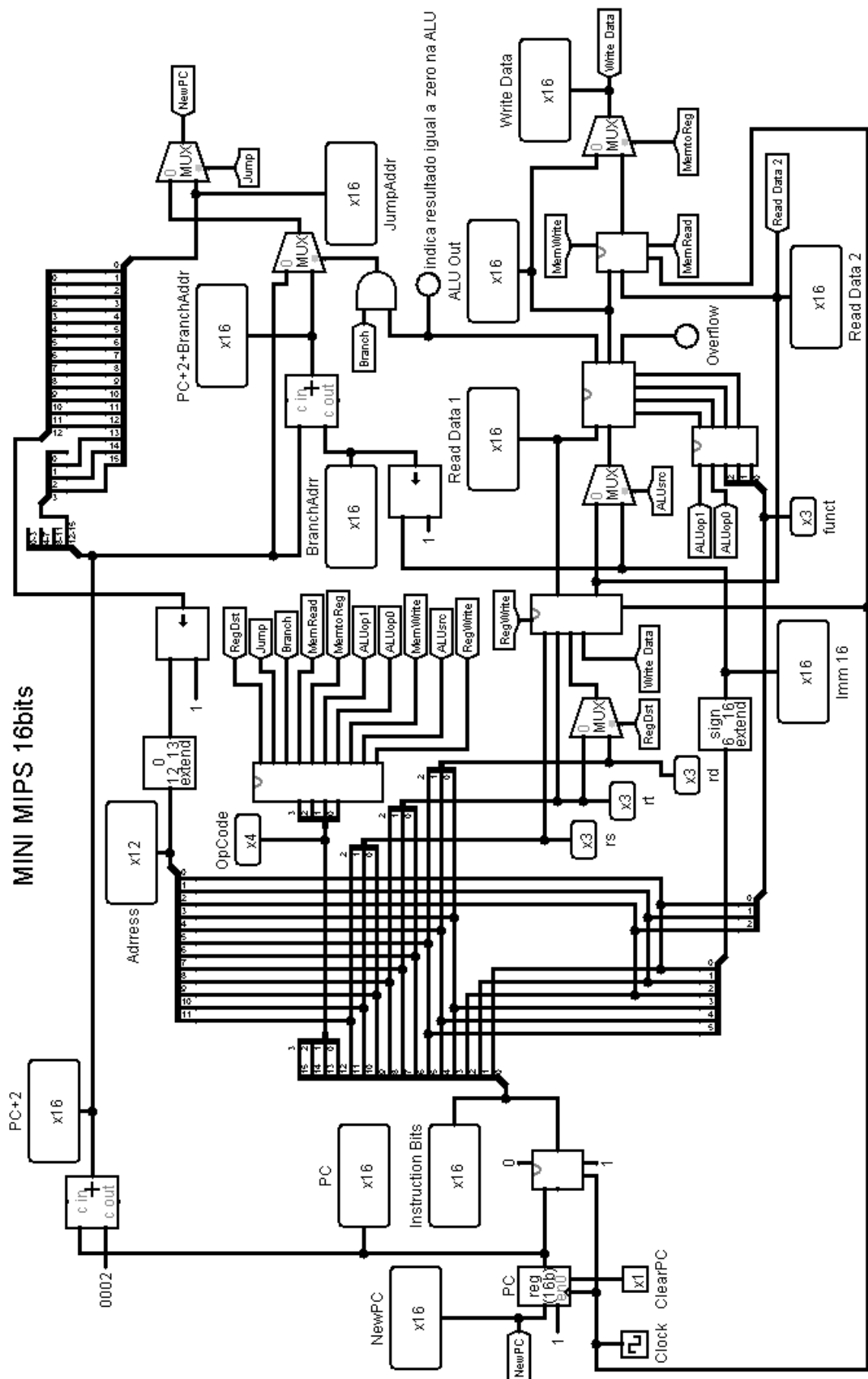


Figure 7: Circuito que implementa o MINI MIPS 16bits